



中山大學
SUN YAT-SEN UNIVERSITY

数据抽象和类 I

中山大学计算机学院



主讲人：郑贵锋



中山大学MOOC课程组

zhenggf@mail.sysu.edu.cn

目录

CONTENTS

01

基本概念

02

结构体、类与对象

03

成员函数



先看第一个例子

Date.cpp



抽象

Data abstraction: 只关心该数据“是什么”及“如何使用”，而不关心它是如何运作的。

Control abstraction: 只关心这个行为能够为我们带来什么，而不关心这个行为的具体实现方法。



抽象数据类型

在程序设计中，对于被抽象的数据，称为**抽象数据类型**
(Abstract Data Type, **ADT**)。

一种ADT应具有

- 1 **说明部分**（说明该该ADT是什么及如何使用）：说明部分描述数据值的特性和作用于这些数据之上的操作。ADT的用户仅须明白这些说明，而无须知晓其内部实现。
- 2 **实现部分**。



抽象数据类型转化

把DATE设计为一种数据类型。

- 内部包含年月日等数据以及在这些数据上可进行的操作。
- 用户利用DATE就可以定义多个变量。
- 用户可调用每个变量中公开的操作，但无法直接访问每个变量中被隐藏的内部数据。
- 用户也无需关心变量中各操作的具体实现。
- 于是DATE就是一种封装好的数据类型。这就达到了信息隐藏和封装的目的。



抽象数据类型大致结构-以日期为例

Type

DATE

Data

Each DATE value is date
in day/month/year

int year, month, day;

Operation

Set the date

Get the date

Increment the date by one day

Decrement the date by one day

**set()
get()
increment()
decrement()**



回顾C语言当中的结构体

C 数组允许定义可存储相同类型数据项的变量，**结构**是 C 编程中另一种用户自定义的可用的数据类型，它允许您存储不同类型的数据项。结构用于表示一条记录，我们继续以上面的日期为例，我们可能会关心：

- Year
- Month
- Day

但C语言中的 struct 只能包含变量，不能包括实现ADT当中操作的函数！

```
struct Date {  
    int year;  
    int month;  
    int day ;  
} date;  
void Set( int, int, int  
);  
int getMonth();  
int getDay();  
int getYear;  
void Print();  
void Increment();  
void Decrement();
```




C++ 对象&类

C++ 在 C 语言的基础上增加了**面向对象编程**，C++ 支持面向对象程序设计。类是 C++ 的核心特性，通常被称为用户定义的类型。

类用于指定对象的形式，它包含了数据表示法和用于处理数据的方法。类中的数据和方法称为类的成员。函数在一个类中被称为类的成员。

定义一个类，本质上是定义一个数据类型的蓝图。这实际上并没有定义任何数据，但它定义了类的名称意味着什么，也就是说，它定义了类的对象包括了什么，以及可以在这个对象上执行哪些操作。

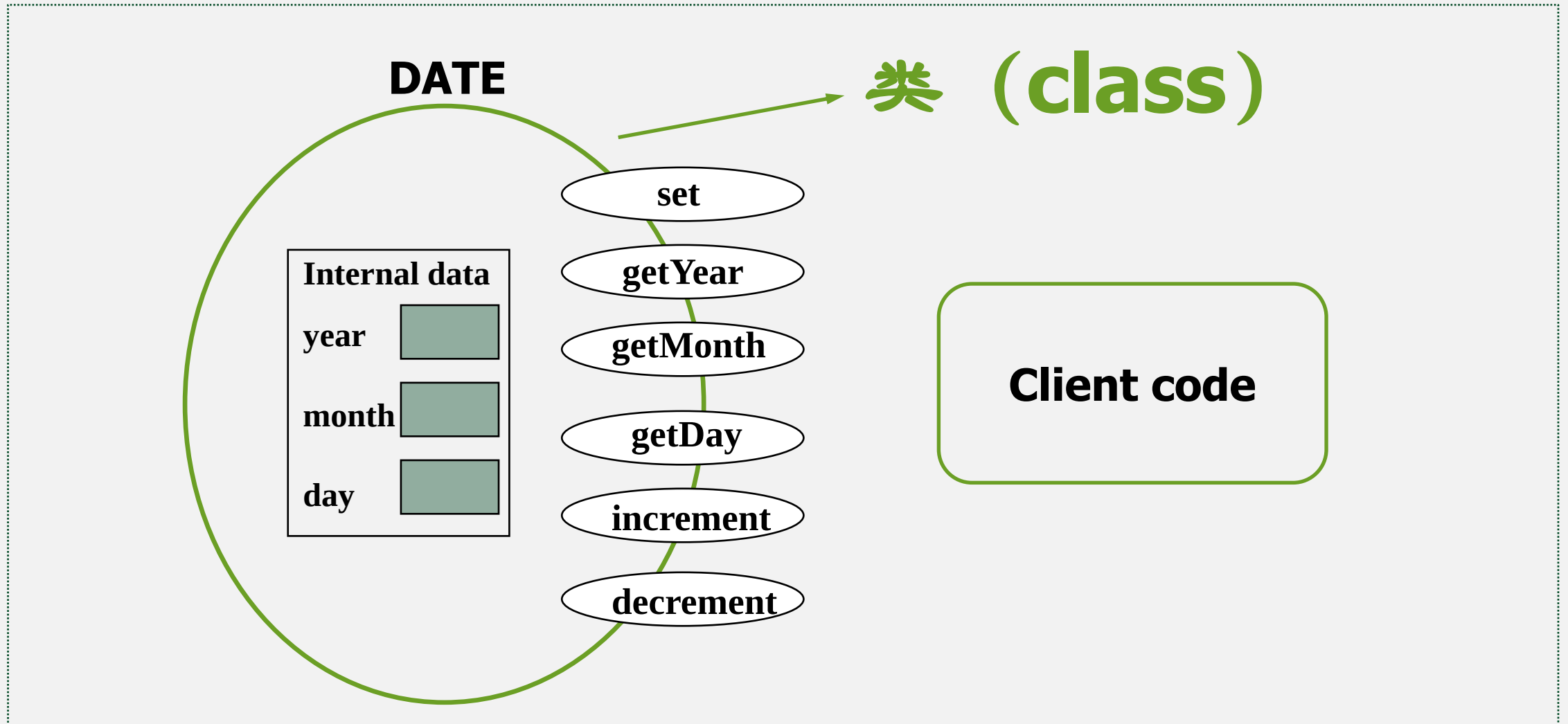
```
class classname  
{  
    Access specifiers: // 访问修饰符: private/public/protected  
    Data members/variables; // 变量  
    Member functions() {} // 方法  
}; // 分号结束一个类
```

The diagram illustrates the C++ class syntax with the following annotations:

- 关键字** (Keyword): Points to the `class` keyword.
- 类名** (Class Name): Points to the `classname` identifier.
- 访问修饰符** (Access Specifier): Points to the `Access specifiers:` label, with examples `private/public/protected`.
- 变量** (Variable): Points to the `Data members/variables;` line.
- 方法** (Method): Points to the `Member functions() {}` line.
- 分号结束一个类** (Semicolon ends a class): Points to the closing semicolon `};`.



Date类





类的例子

类定义是以关键字 **class** 开头，后跟类的名称。类的主体是包含在一对花括号中。类定义后必须跟着一个分号或一个声明列表。例如，我们使用关键字 **class** 定义 Date 数据类型，如右所示。

```
class DATE {  
    public:  
        void Set( int, int, int );  
        int getMonth() const;  
        int getDay() const;  
        int getYear() const;  
        void Print() const;  
        void Increment();  
        void Decrement();  
    private:  
        int month;  
        int day;  
        int year;  
};
```



类的例子

在{}中列出类的成员。类的成员包括：

- **数据成员**：一般说来，数据成员是需要隐藏的对象；即外部的程序是不能直接访问这些数据的，应该通过函数成员来访问这些数据。所以一般情况下，数据成员通过关键字**private**声明为私有成员（private member）
- **函数成员**：通过关键字**public**声明为公有成员（public member）。外部程序可以访问共有成员，但无法访问私有成员。

对于**类的使用者**（即用户代码，简称用户）而言，只需要获得**DATE.h**，即可调用类对象的公有函数访问其内部的数据成员。使用者无法直接访问私有成员，也无需知晓公有函数的内部实现。};

```
class DATE {  
    public:  
        void Set( int, int, int );  
        int getMonth() const;  
        int getDay() const;  
        int getYear() const;  
        void Print() const;  
        void Increment();  
        void Decrement();  
    private:  
        int month;  
        int day;  
        int year;  
};
```



结构体与类

C语言中的 struct 只能包含变量，而 C++ 中的 class 除了可以包含变量，还可以包含函数。set() 是用来处理成员变量的函数，在C语言中，我们将它放在了 struct Date 外面，它和成员变量是分离的；而在 C++ 中，我们将它放在了 class Date 内部，使它和成员变量聚集在一起，看起来更像一个整体。



结构体、类与对象

结构体和类都可以看做一种由用户自己定义的复杂数据类型，在C语言中可以通过结构体名来定义变量，在 C++ 中可以通过类名来定义变量。不同的是，通过结构体定义出来的变量传统上叫变量，而通过类定义出来的变量有了新的名称，叫做对象（Object）。

在右边代码中，我们先通过 class 关键字定义了一个类 Date，然后又通过 Date 类创建了一个对象 date。

```
class DATE {  
    public:  
        void Set( int, int, int );  
        int getMonth() const;  
        int getDay() const;  
        int getYear() const;  
        void Print() const;  
        void Increment();  
        void Decrement();  
    private:  
        int month;  
        int day;  
        int year;  
};  
DATE date;
```



成员函数

类的成员函数是指那些把定义和原型写在类定义内部的函数，就像类定义中的其他变量一样。类成员函数是类的一个成员，它可以操作类的任意对象，可以访问对象中的所有成员。

让我们看看之前定义类 Date，现在我们要使用成员函数来访问类的成员，而不是直接访问这些类的成员。

```
class DATE {  
    public:  
        void Set( int, int, int );  
        int getMonth() const;  
        int getDay() const;  
        int getYear() const;  
        void Print() const;  
        void Increment();  
        void Decrement();  
    private:  
        int month, day, year;  
};
```



成员函数

成员函数可以定义在类定义内部，
或者单独使用**范围解析运算符**
:: 来定义。在类定义中定义的
成员函数把函数声明为**内联**的，
即便没有使用 `inline` 标识符。所以
您可以按照如下方式定
义 `Set( int Y, int M, int D)` 函数。

```
class DATE {  
    public:  
        void Set(int newYear,  
                  int newMonth,  
                  int newDay ) {  
            month = newMonth;  
            day = newDay;  
            year = newYear;  
        }  
        .....  
    private:  
        .....  
};
```




成员函数

也可以在类的外部使用**作用域解析运算符 ::** 定义该函数，如右所示。

```
class DATE {  
    public:  
        void Set(int newYear,  
                  int newMonth,  
                  int newDay );  
  
        .....  
    private:  
        .....  
};  
void DATE::Set(int newYear,  
                int newMonth,  
                int newDay ) {  
  
    month = newMonth;  
    day = newDay;  
    year = newYear;  
}
```



DATE类中成员函数

```
#include "DATE.h"
#include <iostream>
using namespace std;

int DaysInMonth( int, int );

void DATE::Set(int newYear,

int newMonth,

int newDay )
{ }
```

```
int DATE::getMonth() const
{ }
int DATE::getDay() const
{ }
int DATE::getYear() const
{ }
void DATE::Print() const
{ }
void DATE::Increment()
{ }
void DATE::Decrement()
{ }
int DaysInMonth( int mo, int yr )
{ }
```



其他成员函数实现

//DATE.cpp the implementation of each member
//function of DATE.

```
void DATE::Set( int newYear,  
                int newMonth,  
                int newDay ) {  
    month = newMonth;  
    day = newDay;  
    year = newYear;  
}
```



其他成员函数实现

//DATE.cpp the implementation of each member function of
//DATE.

```
int DATE::getMonth() const
{
    return month;
}
```

```
int DATE::getDay() const
{
    return day;
}
```

```
int DATE::getYear() const
{
    return year;
}
```



其他成员函数实现

```
//DATE.cpp the implementation of each  
member function of //DATE.  
void DATE::Print() const  
{  
    switch (month)  
    {  
        case 1 : cout << "January";  
                break;  
        case 2 : cout << "February";  
                break;  
                :  
        case 12 : cout << "December";  
    }  
    cout << ' ' << day << ", " << year <<  
endl << endl;  
}
```



其他成员函数实现

//DATE.cpp the implementation of each member function
of //DATE.

```
void DATE::Increment()
{
    day++;
    if (day > DaysInMonth(month, year))
    {
        day = 1;
        month++;
        if (month > 12)
        {
            month = 1;
            year++;
        }
    }
}
```



其他成员函数实现

//DATE.cpp the implementation of each member function of DATE.

```
void DATE::Decrement()
{
    day--;
    if ( day == 0 )
    {
        if( month == 1 )
        {
            day = 31;
            month = 12;
            year--;
        }
        else
        {
            month--;
            day = DaysInMonth( month, year );
        }
    }
}
```



其他成员函数实现

```
//DATE.cpp the implementation of the auxiliary function
//DaysInMonth.
int DaysInMonth( /* in */ int mo, /* in */ int yr )
{
    switch (mo)
    {
        case 1: case 3: case 5: case 7: case 8: case 10: case 12:
            return 31;
        case 4: case 6: case 9: case 11:
            return 30;
        case 2:
            if ((yr % 4 == 0 && yr % 100 != 0) || yr % 400 == 0)
                return 29;
            else
                return 28;
    }
}
```




成员函数

在DATE.cpp文件开头需要加入预处理命令

```
#include "DATE.hpp"
```

这是因为在DATE.cpp中要用到用户自定义的标识符DATE，而它的定义在DATE.hpp中。在DATE.hpp中，各函数原型是在{}中的。根据标识符的作用域规则，它们的作用范围仅在类定义中，而不包括DATE.cpp。因此在DATE.cpp中需要利用作用域解释运算符“::”来指明这里的函数是类DATE里的成员函数。

DATE.cpp中有时还包括DATE内部要使用到的函数，例如DaysInMonth。这种函数并非对外公开供用户使用，因此可以将其声明为类的私有成员。若在该函数中没有涉及该类的数据成员，则无需将它们声明为类的成员。



访问成员变量和成员函数

调用成员函数和成员变量是在对象上使用点运算符 (.)，这样它就能操作与该对象相关的数据，如下所示：

```
date.flag = 1;  
date.set(2022,1,30);
```

```
class DATE {  
    public:  
        void Set(int newYear,  
                  int newMonth,  
                  int newDay ) {  
            month = newMonth;  
            day = newDay;  
            year = newYear;  
        }  
        int flag;  
        .....  
    private:  
        .....  
}date;
```



访问成员变量和成员函数例子

```
//client.cpp
#include "DATE.h"
#include <iostream>
using namespace std;
int main()
{
    DATE date1, date2; //①
    int tmp;

    date1.Set( 1999, 10,
1 );
    date1.Print();
    date1.Increment();
    date1.Print();

    :

}
```

```
date2.Set( 1997, 7, 1 );
date2.Print();
date2.Decrement();
date2.Print();

tmp = date1.getYear();
tmp++;
date1.Set( tmp, 12, 20 );
date1.Print();

cout << date1.year;

//error?

return 0;

}
```



类的静态成员

- 静态（static）成员是类的组成部分但不是任何对象的组成部分
- 通过在成员声明前加上保留字static将成员设为static（在数据成员的类型前加保留字static声明静态数据成员；在成员函数的返回类型前加保留字static声明静态成员函数）
- static成员遵循正常的公有/私有访问规则。C++程序中，如果访问控制允许的话，可在类作用域外直接（不通过对象）访问静态成员（需加上类名和::）



类的静态成员

- 静态数据成员具有静态生存期，是类的所有对象共享的存储空间，是整个类的所有对象的属性，而不是某个对象的属性。
- 与非静态数据成员不同，静态数据成员不是通过构造函数进行初始化，而是必须在类定义体的外部再定义一次，且恰好一次，通常是在类的实现文件中再声明一次，而且此时不能再用static修饰。



类的静态成员

- 静态数据成员具有静态生存期，是类的所有对象共享的存储空间，是整个类的所有对象的属性，而不是某个对象的属性。
- 与非静态数据成员不同，静态数据成员不是通过构造函数进行初始化，而是必须在类定义体的外部再定义一次，且恰好一次，通常是在类的实现文件中再声明一次，而且此时不能再用static修饰。
- 静态成员函数不属于任何对象
- 静态成员函数没有this指针
- 静态成员函数不能直接访问类的非静态数据成员，只能直接访问类的静态数据成员



类的静态成员

```
class DATE          // DATE.h
{
public:
    DATE( int =2000, int =1, int = 1);

    static void getCount( );

    void Set( int, int, int);
    int getMonth() const;
    int getDay() const;
    int getYear() const;
    void Print() const;
    void Increment();
    void Decrement();
    :
```

```
    :
private:
    int month;
    int day;
    int year;
    static int count;
};
```



类的静态成员

```
//DATE.cpp      StaticMember  
int DATE::count = 0; //必须在类定义体的外部再定义一次
```

```
DATE::DATE( int initYear, int initMonth, int initDay )  
{  
    year = initYear;  
    month = initMonth;  
    day = initDay;  
    count++;  
}  
void DATE::getCount()  
{  
    cout << "There are " << count << " objects now" << endl;  
}
```

```
void main()  
{  
    DATE DATE1;  
    DATE DATE2( 1976, 12, 20 );  
  
    DATE::getCount();  
}
```




中山大學
SUN YAT-SEN UNIVERSITY

谢谢

中山大学计算机学院



中山大学MOOC课程组